COMP2022 Models of Computations: Predicate Logic

Contents

1 Introduction **2**

2 Domains **2**

3 Predicates **3**

4 Quantifiers **4**

5 Translating to and from logic **5**

5.1 Tips (i) 5

5.2 Tips (ii) 6

6 Equivalences involving quantifiers **6**

7 Bound/Free variables **7**

8 Syntax and semantics **7**

8.1 Syntax 7

8.2 Semantics 9

9 Validity **10**

10 Good to know **11**

1 Introduction

Can we formalise these arguments in propositional logic?

- "Romeo is a man"
- "All men are mortal"
- So, Romeo is mortal

Propositional Logic does not have a way to talk about:

1. individuals/objects (Romeo),
2. that certain objects have properties (e.g. being a man, being mortal),
3. or that a property holds for all individuals/objects (all men are mortal)

Predicate-logic (aka **First-order logic**) is for modelling

1. **objects** (like numbers)
2. **properties** of objects ('x is even')
3. **relations** between objects (like 'x is greater than y')

It generalises Propositional Logic. The main new ingredients are:

1. Domain: collection of objects
2. Predicates: properties of objects and relations between objects
3. Variables: vary over objects in the domain
4. Quantifiers: allow one to reason about multiple objects

2 Domains

When we reason aobut objects, we have in mind a certain domain of discourse.

1. In programming, the domain may be the integers, or strings, or both, etc.
2. In the world, the domain includes people, animals, etc.

Definition. A *domain* is a non-empty set $\mathbb{D}$. It's elements are sometimes called *objects*.

- The set $\mathbb{Z}$ of integers is a domain, e.g. it contains -3.
- The set $\mathbb{H}$ of humans is a domain, e.g. it contains someone called `Romeo`.
- The set $\mathbb{S}$ of binary strings is a domain, e.g. it contains " `011`"

Variables $x, y, z, \ldots$ vary over elements of the domain.

3 Predicates

In propositional logic we write propositions:

- 'Romeo is happy'
- 'Romeo loves Juliet'

In predicate logic we use predicates:

- `happy(Romeo)`
- `loves(Romeo, Juliet)`

With variables we can also write *atomic formulas*:

- `happy(`x`)`
- `loves(`x`, `y`)`
- `loves(Romeo, `y`)`

Using connectives, we can now write some simple formulas:

- `loves(Romeo,Juliet)` $\rightarrow$ `happy(Juliet)`
- `loves(Romeo, `x`)` $\rightarrow$ `happy(`x`)`

Definition. A *predicate* (of arity k) over domain $\mathbb{D}$ is a subset of $\mathbb{D}^k$.

- `happy` $\subseteq \mathbb{H}$ is a unary predicate ($k = 1$)

- $\texttt{loves} \subseteq \mathbb{H} \times \mathbb{H}$ is a binary predicate ($k = 2$)

We also allow infix notation:

$$x \,\texttt{loves}\, y$$

We also allow functional notation:

$$\texttt{loves} : \mathbb{H} \times \mathbb{H} \to \{\texttt{true}, \texttt{false}\}$$

- Arguments in atomic formulas are variables and objects.

$$\texttt{loves}(x, \texttt{Juliet})$$

- If we fix the values of the variables, then predicates become propositions. And so are either true or false.

$$\texttt{loves}(\texttt{Romeo}, \texttt{Juliet})$$

- We cannot compose predicates. E.g.

$$\texttt{happy}(\texttt{loves}(x, y))$$

  is not a formula and has no meaning.

4 Quantifiers

There are two types of quantifiers.

1. The **existential quantifier**, $\exists$.

$$\exists x F$$

   is true if there is an element d of the domain so that F true when d replaces x in F.

2. The **universal quantifier**, $\forall$.

$$\forall x F$$

   is true if for every element d of the domain, the formula F is true when d replaces x in F.

A quantified formula

$$\exists x F$$

has two parts:

1. The variable being quantified x
2. The formula being quantified F

We can nest quantifiers: $\exists x \exists y F$ and even mix them $\exists x \forall y F$.

Example. Consider domain $\mathbb{H}$ of humans and the predicate $\texttt{loves}(x, y)$:

- $\forall x\, \texttt{loves}(x, x)$ means "Every human loves themselves"
- $\forall x \forall y\, \texttt{loves}(x, y)$ means "Everyone loves everyone"
- $\exists x \forall y\, \texttt{loves}(x, y)$ means "Someone loves everyone"
- $\forall x \exists y\, \texttt{loves}(x, y)$ means "Everyone loves someone"
- $\exists x \exists y\, \texttt{loves}(x, y)$ means "Someone loves someone"

5 Translating to and from logic

Think of logic as a programming language that is based in mathematics.

- Programming languages (datalog, answer set programming)
- Database query languages (SQL)
- Hoare logic for verifying correctness of programs

5.1 Tips (i)

There are some common forms:

1. "All As are Bs" translates as $\forall x(A(x) \to B(x))$
2. "Some As are Bs" translates as $\exists x(A(x) \wedge B(x))$
3. "No As are Bs" translates as $\forall x(A(x) \to \neg B(x))$
4. "Some As are Bs" translates as $\exists x(A(x) \wedge \neg B(x))$

Usually: $\wedge$ goes with $\exists$; $\to$ goes with $\forall$.

Example. For domain $\mathbb{Z}$, predicates `even`, `odd` and `greater`. Translate the statement "Every even integer is greater than some odd integer" into predicate logic.

- This is of the form "All As are Bs"
- $A(x)$ for "x is an even integer"

- $B(x)$ for "x is greater than some odd integer"

$$\forall x(\texttt{even}(x) \to \exists y(\texttt{odd}(y) \land \texttt{greater}(x, y)))$$

Translate the statement "Some even integer is equal to 0" into predicate logic.

1. $\exists x(\texttt{even}(x) \land \texttt{equal}(x, 0))$
2. $\exists x(\texttt{even}(x) \to \texttt{equal}(x, 0))$

Translate the statement "Every even integer is equal to 0" into predicate logic.

1. $\forall x(\texttt{even}(x) \land \texttt{equal}(x, 0))$
2. $\forall x(\texttt{even}(x) \to \texttt{equal}(x, 0))$

5.2 Tips (ii)

The order of quantifiers only matters when mixing existential and universal quantifiers.

- $\forall x \forall y P(x, y)$ means the same thing as $\forall y \forall x P(x, y)$
- $\exists x \exists y P(x, y)$ means the same thing as $\exists y \exists x P(x, y)$
- $\exists y \forall x P(x, y)$ means there is a single y such that for all x we have that $P(x, y)$ is true.
- $\forall x \exists y P(x, y)$ means for every x there is a y (that can be different for different choices of x) such that $P(x, y)$ is true.

6 Equivalences involving quantifiers

For all formulas F, G:

(Q. Negation)	$\neg \forall x F \equiv \exists x \neg F$
	$\neg \exists x F \equiv \forall x \neg F$
(Q. Unification)	$(\forall x F \land \forall x G) \equiv \forall x (F \land G)$
	$(\exists x F \lor \exists x G) \equiv \exists x (F \lor G)$
(Q. Transposition)	$\forall x \forall y F \equiv \forall y \forall x F$
	$\exists x \exists y F \equiv \exists y \exists x F$
(Q. Extraction)	if $x \notin \text{Free}(G)$:
	$(\forall x F \land G) \equiv \forall x (F \land G)$
	$(\forall x F \lor G) \equiv \forall x (F \lor G)$
	$(\exists x F \land G) \equiv \exists x (F \land G)$
	$(\exists x F \lor G) \equiv \exists x (F \lor G)$

7 Bound/Free variables

An occurrence of the variable x in the formula F is **bound** if x occurs within a subfomula of F of the form $\exists x G$ or $\forall x G$. Otherwise it is a **free** occurrence.

- This is similar to local variables and global variables in programming.
- A variable may have both free and bound occurrences in a formula F.
- Intuitively, to get a proposition from a formula we need to instantiate all the free variables.
- A formula without free variables is called a *sentence*.

8 Syntax and semantics

To give the precise syntax and semantics of predicate logic, we need to separate the vocabulary from the domain/structure.

- First-order structure
- Vocabulary

A **first-order structure** (aka **structure**) consists of a domain $\mathbb{D}$, predicates on $\mathbb{D}$, and constants from $\mathbb{D}$.

e.g., $(\mathbb{Z}, \texttt{plus}, \texttt{0})$
e.g., $(\mathbb{S}, \texttt{plus}, \texttt{0})$

We will use superscripts to distinguish predicates.
e.g., $\texttt{plust}^{\mathbb{Z}}$ is addition on integers
e.g., $\texttt{plus}^{\mathbb{S}}$ is concatenation on strings ($\texttt{0}^{\mathbb{Z}}$ is the integer 0, while $\texttt{0}^{\mathbb{S}}$ is the empty-string).

We distinguish between `plus` as a symbol, and as a relation on a specific domain.

A **vocabulary** (aka **signature**) is a collection of **symbols**, i.e., predicate symbols and constant symbols (that are used as **names** for the predicates and constants in structures).

E.g., the vocabulary of both the structures $(\mathbb{Z}, \texttt{plus}, \texttt{0})$ and $(\mathbb{S}, \texttt{plus}, \texttt{0})$ have a single predicate `plus` and a single constant `0`.

8.1 Syntax

Fix a vocabulary.

Definition. A *term* is a variable $x, y, z, \ldots$ or a constant symbol $c, d, e, \ldots$

An *atomic formula* has the form $P(t_1, \ldots, t_k)$ where P is a k-ary predicate symbol and $t_1, \ldots, t_k$ are terms.

A *formula* is defined by the following recursive process:

1. Every atomic formula is a formula.
2. If F is a formula then $\neg F$ is a formula.
3. If F, G are formulas then $(F \wedge G)$ and $(F \vee G)$ are formulas.
4. If F is a formula and x a variable then $\exists x F$ and $\forall x F$ are formulas.

Example.

- Vocabulary
  - Binary predicate symbol `greater_or_equal`
  - Constant symbols 0, 1
- Domain $\mathbb{Z}$
- Terms: $x, y, z, 0, 1$
- Atomic formulas
  - $\texttt{greater_or_equal}(x, y)$
  - $\texttt{greater_or_equal}(x, 0)$
  - $\texttt{greater_or_equal}(x, x)$
- Formulas:
$$\forall x \, (\texttt{greater_or_equal}(x, 0) \vee \texttt{greater_or_equal}(x, y))$$

What about semantics of predicate logic?

- The truth value of a formula obviously depends on the structure it is interpreted over.
- But it also depends on the values of the variables.

An **assignment** maps variables to objects (elements of the domain)

- Typically denote assignments by the letter α
- So if F is a formula and α an assignment then $\alpha(F)$ is either true or false

Example.

- Domain $\mathbb{Z}$
- Assignment $\alpha(x) = 3$, $\alpha(y) = 2$
- The following formulas are true:
  1. $\texttt{greater_or_equal}(x, y)$ (aka $x \geq y$)
  2. $\texttt{greater_or_equal}(y, y)$ (aka $y \geq y$)

8.2 Semantics

Fix a vocabulary and a structure with domain $\mathbb{D}$.

Definition. The *truth-value of a formula F under assignment α* is defined recursively:

1. Predicates:
   (a) Unary predicate P is true under α if $\alpha(x) \in P$.
   (b) Binary predicate Q is true under α if $(\alpha(x), \alpha(y)) \in Q$.
2. The truth-value of the Boolean connectives are as usual.
3. $\forall x F$ is true under α if for every $d \in \mathbb{D}$, F is true under $\alpha[x := d]$.
4. $\exists x F$ is true under α if there is some $d \in \mathbb{D}$ such that F is true under $\alpha[x := d]$.

Here $\alpha[x := d]$ is the assignment that is identical to α except that it maps x to d. This is like replacing x by d.

Note the recursion in this definition.

To see if $\forall x F$ is true under α we must see if the simpler formula F is true under a bunch of assignments, i.e., $\alpha[x := d]$ for every $d \in \mathbb{D}$.

Example.

- Domain $\mathbb{Z}$
- Formula $\forall x (x \geq y)$
- Assignment $\alpha(y) = 0$ (and we don't care what the value of α on x is, since x is bound).

Let's apply the definition to show that the formula is false under the assignment.

- We want to know if $\forall x(x \geq y)$ is true under α.
- Same as $x \geq y$ being true under $\alpha[x := d]$, for every integer d.
- Same as $\alpha[x := d](x) \geq \alpha[x := d](y)$ for every integer d.
- Same as $d \geq 0$ for every integer d.
- This is false about the integers, e.g., take $d = -3$.
- Conclude that $\forall x(x \geq y)$ is false under α.

9 Validity

Fix a vocabulary.

A formula F is **valid** if it evaluates to true for every structure and assignment.

Examples.

- $\forall x(P(x) \vee \neg P(x))$ is valid.
- $\forall x \exists y\, Q(y, x)$ is not valid since it is not true statement about the natural numbers with $Q(y, x)$ meaning $y < x$.

Does the following argument make logical sense?

1. All tall people are happy
$$\forall x(T(x) \to H(x))$$
2. There is someone who is happy.
$$\exists x H(x)$$
3. So, there is someone who is tall.
$$\exists x T(x)$$

We can show that
$$(\forall x(T(x) \to H(x)) \wedge \exists x H(x)) \to \exists x T(x)$$
is not valid by finding a **counterexample**, i.e., domain and predicates that make it false.

	$T(x)$	$H(x)$
`Alan`	`0`	`1`
`Bob`	`0`	`0`

How do we show that a formula of predicate logic is valid?

- For propositional logic we could use truth-tables or deduction.
- For predicate logic, we can use a proof-system like Natural Deduction. This allows us to prove all (and only) the validaties.

10 Good to know

Is there an algorithm that decides if a given predicate logic sentence is valid?

- The language
$$\{\langle F \rangle : F \text{ is a valid predicate-logic formula}\}$$
is Turing-recognisable (since we have a sound and complete proof system) but not Turing-decidable.
- Intuitively, this means that no algorithm can decide logical truth.

Predicate logic can also include functions

$$f : \mathbb{D}^k \to \mathbb{D}$$

For instance, in the domain of integers

$$\texttt{plus} : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$$

$$\texttt{square} : \mathbb{Z} \to \mathbb{Z}$$

These can be written using infix notation.

Functions allow us to write **terms** that are more complex than simply variables and elements of the domain, e.g., $x^2 + 3$.

Terms can then be arguments in predicates, e.g., $\texttt{even}(x^2 + 3)$.